Estructuras de Datos

2026
Primer Cuatrimestre

Challenge 2026 Universitario

Smartport Lab | PUERTO BAHÍA BLANCA

BAHIA HUB

UNS

UTN BHI

Conectando el talento universitario con el ecosistema portuario-industrial

¿Qué es el Challenge Universitario 2026?

El Challenge Universitario 2026 convoca a estudiantes universitarios de grado y pregrado a trabajar sobre **desafíos** reales **del ecosistema portuario–industrial** de Bahía Blanca.

Un espacio para desarrollar soluciones innovadoras, **con mentalidad emprendedora y foco en impacto.**

Vas a trabajar en equipo, con acompañamiento y espacios de intercambio.

- **Duración: 2 meses**
- **Metodología: aprender haciendo**
- **Enfoque: innovación, creatividad y mirada emprendedora**
- **Inscripción: equipos de 2 a 4 integrantes** (Un integrante realiza la postulación por todo el equipo)
- **Premios: $10.000.000** para los equipos ganadores

¿Cómo funciona?

Conocé las etapas del challenge y las fechas clave del proceso



¿Qué GANÁS?

- Formación práctica en **innovación y emprendedurismos**
- **Mentorías y acompañamiento** de referentes del ecosistema
- Apoyo para **desarrollar, testear** y hacer crecer tu idea
- **Certificación oficial**
- **$10.000.000 en premios para los equipos ganadores:**
  1° Premio por equipo: $5.000.000
  2° Premio por equipo: $3.000.000
  3° Premio por equipo: $2.000.000
  Más **menciones especiales**
- Viabilidad de **continuar el proyecto** y explorar instancias futuras de implementación o vinculación

REPASO

Motivación - Estructuras de Datos

Desde un punto de vista conceptual, todo lo que mantiene unido a ciertos datos conforma una estructura de datos

Motivación - Estructuras de Datos

Desde un punto de vista conceptual, todo lo que mantiene unido a ciertos datos conforma una estructura de datos

Una fecha es una estructura de datos

11 JUL 1999

Aclaración

```
@Override
public boolean metodoY(int y) {
     return removeIf(filter, 0, size);
}
```



Es una directiva al compilador
No solo favorece la documentación sino que también el compilador chequea que esté haciendo una redefinición

Conceptos de POO

Concepto Nuevo: Interfaz

Conceptos de POO

Concepto Nuevo: Genericidad Paramétrica

Genericidad

Problema:

- Quiero programar una clase Colección una única vez y luego utilizarla para programar una colección de enteros, una colección de caracteres, una colección de personas, etc.

¿Cómo hago esto?

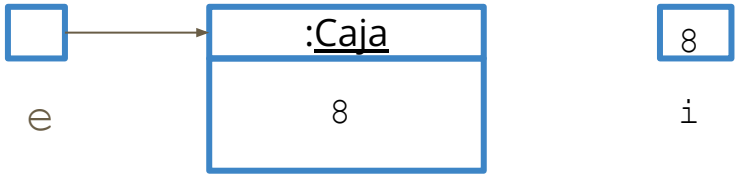
- 1er requerimiento: Los valores de los tipos básicos no se almacenan como objetos y necesito tratar a todos los tipos de datos como objetos.
- 2do requerimiento: Quiero reusar la mayor cantidad de código posible

Manejar tipos básicos como objetos: Encapsular un entero en un objeto

```
public class Caja {
      private int valor;
      public Caja(int x) { valor = x; }
      public int getValor() { return valor; }
      public void setValor( int nuevo_valor ) { valor = nuevo_valor; }
}
```

Entonces para usar estos enteros habría que hacer algo así:

```
Caja e = new Caja(8);
int i = e.getValor();
```



¿Cuál es el problema con este enfoque?

Necesitamos una caja para enteros, otra para float, otra para Personas

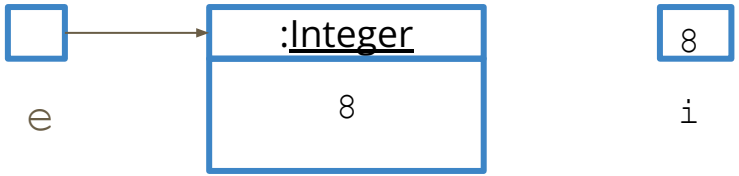
Clases wrapper

- Las clases wrapper existentes:
  - Byte para byte,
  - Short para short,
  - Integer para int,
  - Long para long,
  - Boolean para boolean,
  - Float para float,
  - Double para double
  - Character para char

- Las clases wrapper se usan como cualquier otra:

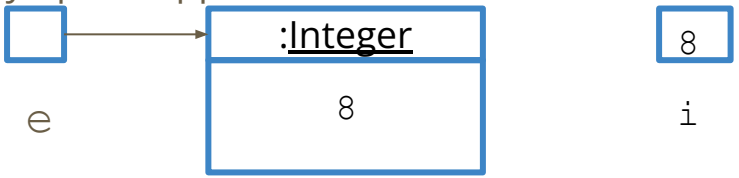
```
Integer e = new Integer(8);//Tienen constructor
int i = e.intValue();//Tienen getters: i contiene un 8
```

- Los objetos de clases wrapper son inmutables
  - no permiten actualizar los valores envueltos porque las clases wrapper no tienen setters
  - Preferencia de uso de autoboxing y unboxing



Nuevos compiladores: Autoboxing y Unboxing

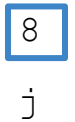
- Autoboxing: Conversión automática entre tipo primitivo y tipo wrapper
  - El valor de *i* se convierte en Integer
  - Creación de objeto, no asignación




```
int i = 8;

Integer e = i;
```

- Unboxing: Conversión automática entre tipo wrapper y tipo primitivo
  - El valor en *e* se extrae y almacena en *j*




```
int j = e;
```

- Los objetos wrapper son inmutables (su valor no cambia)
- Entonces, las clases wrapper no sirven para simular pasaje de parámetros por referencia con tipos primitivos porque Java tiene pasaje de parámetros por valor (ver práctica 1)
- Recordar: Los tipos primitivos tampoco se pueden pasar por referencia

Genericidad con herencia (concepto de IPOO)

¿Cómo generalizo la idea de la caja que contenga cualquier tipo?

```
public class CajaVieja {
      private Object dato;
      public CajaVieja (Object dato) { this.dato = dato; }
      public Object getDato() { return dato; }
}
```

En un programa cliente:

```
CajaVieja cajaEntero = new CajaVieja(8);
int x = cajaEntero.getDato();
```



error de compilación

Genericidad con herencia (concepto de IPOO)

¿Cómo generalizo la idea de la caja que contenga cualquier tipo?

```
public class CajaVieja {
      private Object dato;
      public CajaVieja (Object dato) { this.dato = dato; }
      public Object getDato() { return dato; }
}
```

En un programa cliente:

```
CajaVieja cajaEntero = new CajaVieja(8);
int x = (Integer) cajaEntero.getDato();
```



```
CajaVieja cajaString = new CajaVieja("Hola");
String s = (String) cajaString.getDato();
```



```
int y = (Integer) cajaString.getDato();
```



error tiempo ejecución (ClassCastException)

Genericidad con herencia (concepto de IPOO)

¿Cómo generalizo la idea de la caja que contenga cualquier tipo?

```
public class CajaVieja {
      private Object dato;
      public CajaVieja (Object dato) { this.dato = dato; }
      public Object getDato() { return dato; }
}
```

En un programa cliente:

```
CajaVieja cajaEntero = new CajaVieja(8);
int x = (Integer) cajaEntero.getDato();
```



```
CajaVieja cajaString = new CajaVieja("Hola");
String s = (String) cajaString.getDato();
```



```
cajaEntero = cajaString;
```



compila y ejecuta (tirará error si se busca hacer getDato y castear a entero). Type unsafe

Genericidad con polimorfismo paramétrico

- Java 5.0 incluye un framework de genericidad para usar tipos de dato abstracto que evitan casting explícito
- Un tipo genérico es un tipo que no es definido en compilación sino en tiempo de ejecución
    - permite definir una clase en función de parámetros formales de tipo que abstraen los tipos de algunos datos internos de una clase
- Dada una clase genérica, vamos instanciar objetos usando parámetros actuales de tipo

Genericidad paramétrica

Actualicemos la CajaVieja de Object

```
public class Caja<E> {
      private E dato;
      public Caja(E dato) { this.dato = dato; }
      public E getDato() { return dato; }
      public void setDato( E nuevo_valor ) {
            dato = nuevo_valor; }
}
```

- E se refiere a un tipo en particular
- El constructor toma un objeto de tipo E
- La función getDato devuelve un objeto de tipo E

Para usar esta clase E debe ser reemplazado con una clase específica:

```
Caja<Persona> cp = new Caja<Persona>( p );
// cp es una caja de una persona

Caja<Float> cf = new Caja<Float>(4.5f); //
cf es una caja de un número real

Caja<String> cs = new Caja<String>("pepe");
// cs es una caja de un string
```

Comparación

```
CajaVieja cajaEntero = new CajaVieja(8);
int x = (Integer) cajaEntero.getDato();
```

```
CajaVieja cajaString = new CajaVieja("Hola");
String s = (String) cajaString.getDato();
```

```
int y = (Integer) cajaString.getDato();
```



```
cajaEntero = cajaString;
```



```
Caja<Integer> cajaInt =  new Caja<Integer>(8);

int x = cajaInt.getDato();  // No se necesita cast

Caja<String> cajaStr =  new Caja<String>("hola");

String s = cajaStr.getDato();  //No se necesita
cast
```

Las siguientes sentencias dejan de compilar:

```
String s = (String) cajaInt.getDato();

int y = (Integer) cajaStr.getDato();

cajaInt = cajaStr;
```

Los errores que antes teníamos en ejecución ahora se identifican en compilación

Genericidad con polimorfismo paramétrica
Algunas observaciones

Una clase puede tener múltiples parámetros:

```
class Cosas<A, B, C> { … }
```

Se puede hacer subclassing de clases parametrizadas:

```
class CajaEntero extends Caja<Integer> { … nuevos métodos ….}
```

Se puede extender un tipo parametrizado:

```
class CajaEspecial<E> extends Caja<E> { … nuevos métodos….}
```

La siguiente sentencia es legal:

```
Caja<String> cs = new CajaEspecial<String>(“hola”)
```

Porque caja especial es subclase de caja.

Genericidad con polimorfismo paramétrica
Algunas observaciones (2)

Una clase parametrizada es como cualquier otra clase.

Puede ser usada como tipo de parámetros formales y como tipo de retorno de funciones:

```
Caja<String> método(int i, Caja<Integer> c) { … }
```

Con la caja, puedo simular pasaje por referencia con tipos inmutables:

```
void metodo2( Caja< Integer> c ) {
    // Modifico el estado interno de c con un setter:
    c.setDato(8);
}
```

Genericidad con polimorfismo paramétrica

Problema: Programar una clase para representar un par ordenado que permita almacenar dos valores de cualquier tipo.

• La clase será genérica, se llamará Pair y estará parametrizada en términos de los tipos de las dos componentes del par.

• (Solución presentada en el libro)

```java
/* Implementa un par ordenado (clave, valor) */
public class Pair<K,V>   // K y V son parámetros formales de tipo.
{
    // Los parámetros formales de tipo se pueden usar
    // para declarar atributos y variables:
    protected K key;   // key es tipo K
    protected V value;   // value es de tipo V

    // Los parámetros formales de tipo se pueden usar como
    // tipo en un parámetro formal:
    public Pair(K key, V value) {
        this.key = key;
        this.value = value;
    }

    // Los parámetros formales de tipo se pueden usar como tipo
    // de retorno de una función
    public K getKey() { return key; }
    public V getValue() { return value; }
    public String toString() {
        return "[" +  getKey() +
        "," + getValue() + "]";
    }
}
```

Usando la clase pair

- Crear un par de tipo (string, número real) y le almacenaremos el par (“Sergio”, 3.14):

```
Pair<String, Float> p = new Pair<String,Float>("Sergio", 3.14f);
```

- Crear un par de tipo (entero, carácter) y le almacenaremos el par (8, ‘b’):

```
Pair<Integer, Character> q = new Pair<Integer, Character>(8, 'b’);
```

- Crear un par de tipo (persona, persona) y le almacenaremos a Juan de 80 kg como primera componente y a Tito de 100kg años como segunda:

```
Pair<Persona, Persona> r = new Pair<Persona, Persona>(new Persona("Juan", 80.0f),
new Persona("Tito", 100.0f));
```

- Nota: Usar tipos wrapper en la declaración y no tipos básicos.
  - 3.14f de tipo float se convierte automáticamente en Float.

Conjunto con genericidad con polimorfismo paramétrico

- **Problema**: Implementar una colección genérica
  - Utilizaremos como estructura de datos subyacente un arreglo de elementos de tipo E
  - Crearemos un conjunto de enteros con el contenido { 1, 2 }
  - Crearemos otro conjunto de caracteres con los elementos { a, b }

- Usar una **interfaz** para especificar las operaciones en forma **independiente de las posibles implementaciones**
  - De esta forma esta colección en un tipo de dato abstracto (solo especifico operaciones)

Interfaz *coleccion*

```
//Archivo: Coleccion.java (para completar con más operaciones):
//Note que la interface es genérica, tiene un parámetro formal de tipo E
//que indica el tipo de los elementos de la colección.
public interface Coleccion<E> {
        //Permite insertar item al final de la colección.
        public void insertar(E item);
        //Permite recuperar el elemento i-esimo de la colección
        public E recuperar(int i);
}
```

Clase ColeccionConArreglo<E>

```
// Archivo fuente: ColeccionConArreglo.java
public class ColeccionConArreglo<E> implements Coleccion<E>
{
        protected int cant; // cantidad de elementos almacenados
        protected E [] datos; // arreglo de elementos
        public ColeccionConArreglo(int tamanioMaximo) {
                cant = 0; // Hay 0 elementos al crear la colección
                datos = (E []) new Object[tamanioMaximo]; // produce warning
        }
        public void insertar(E item) {
                if( cant == datos.length ) {
                        // Incrementar tamaño del arreglo datos
                        // Tarea para el lector
                }
                datos[cant++] = item;
        }

        // Precondición: i está entre 0 y la cantidad de elementos-1
        public E recuperar(int i) {
                return datos[i];
        }
}
```

¿Cómo hace el cliente para ver cuántos elementos hay en la colección?

¿Qué ocurre si el cliente no envía un “i” en rango adecuado?

Test

```
public class Test
{
	public static void main( String [] args ) {
		// Creo una colección de enteros.
		// Debo usar la clase wrapper Integer porque los parámetros actuales de tipo no pueden ser tipos básicos.
		// Note como asignamos un objeto de tipo clase  ColeccionConArreglo a una variable de tipo interfaz Colección
		Coleccion<Integer> conj_int = new ColeccionConArreglo<Integer>(5);
		conj_int.insertar( 1 ); // autoboxing de int a Integer
		conj_int.insertar( new Integer(2) ); // Alternativa redundante
		int i = conj_int.recuperar( 1 ); // Sin casting!
		System.out.println( "x: " + i ); // Imprime x: 2

		// Creo una colección de caracteres
		Coleccion<Character> conj_ch =
		new ColeccionConArreglo<Character>(5);
		conj_ch.insertar( 'a' ); // Autoboxing de char a Character
		conj_ch.insertar( 'b' );
		char c = conj_ch.recuperar( 1 );
		// Sin casting! + unboxing de Character a char ( recuperar retorna un Character que es convertido a char para hacer la
asignación).
		System.out.println( "c: " + c ); // Imprime: c : b
	}
}
```

TDA



PLAYLIST

sunflower

rex orange county, summer salt, girl in red, daniel caesar, peachy!, billie eilish, the beatles, cavetown, clairo, peach pit, tyler the creator,...

Created by mhl343 • 186 songs, 10 hr 45 min

PLAY FOLLOWING ...

FOLLOWERS
575

Filter

Download

	TITLE	ARTIST	ALBUM	
+	Drop the Guillotine	Peach Pit	Sweet FA	2018-07-23
+	Wine and Roses	Peachy!	Floral EP	2018-07-23
✓	summer depression	girl in red	summer depre...	2018-07-23

TDA

T = Tipo

D = Dato

A = Abstracto

(ADT Abstract Data Type)

TDA

T = Tipo

D = Dato

A = Abstracto

Todo TDA se define en términos de su representación conceptual y sus operaciones (y de las restricciones que valen entre ellas)

TDA

T = Tipo

D = Dato

A = Abstracto

Una Clase **ES** un TDA

Pero también puede incluir código

TDA Pila / ADT Stack

TDA Pila / ADT Stack



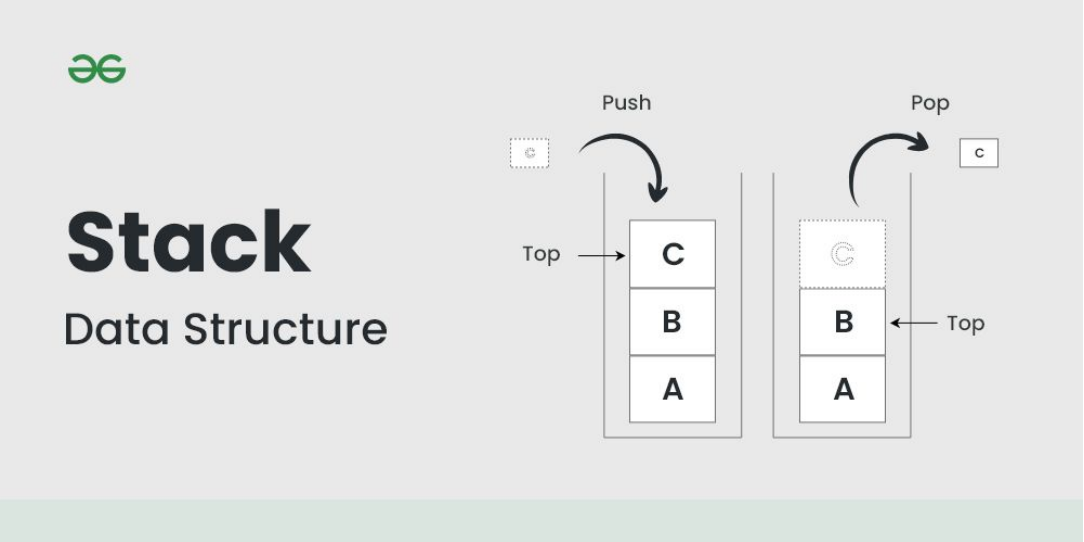
TDA Pila / ADT Stack



Si alguien necesita un plato limpio, lo saca del tope de la pila.

Cuando alguien termina de lavar un plato lo pone en el tope de la pila.

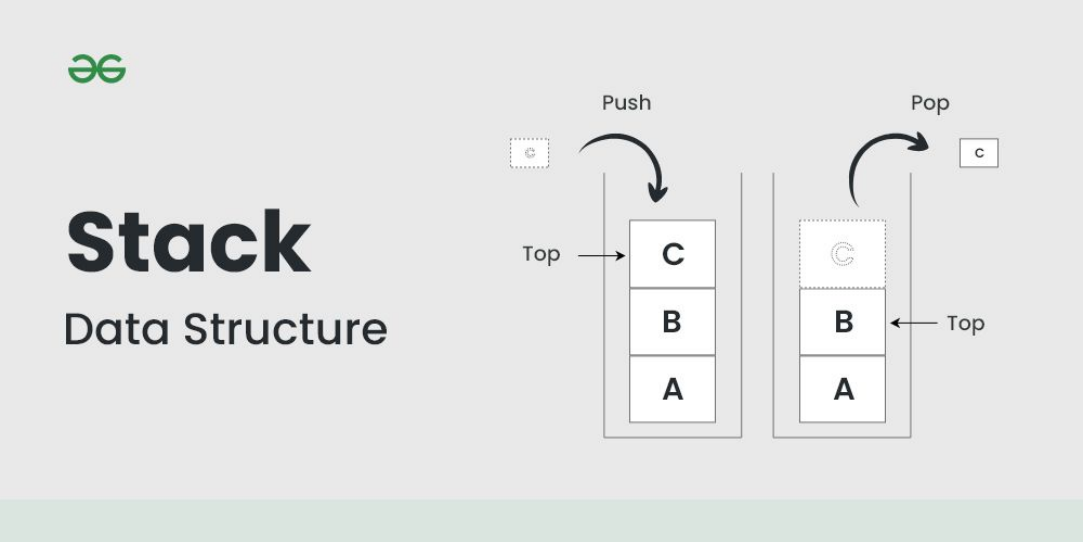
TDA Pila / ADT Stack



Si alguien necesita un plato limpio, lo saca del tope de la pila.

Cuando alguien termina de lavar un plato lo pone en el tope de la pila.

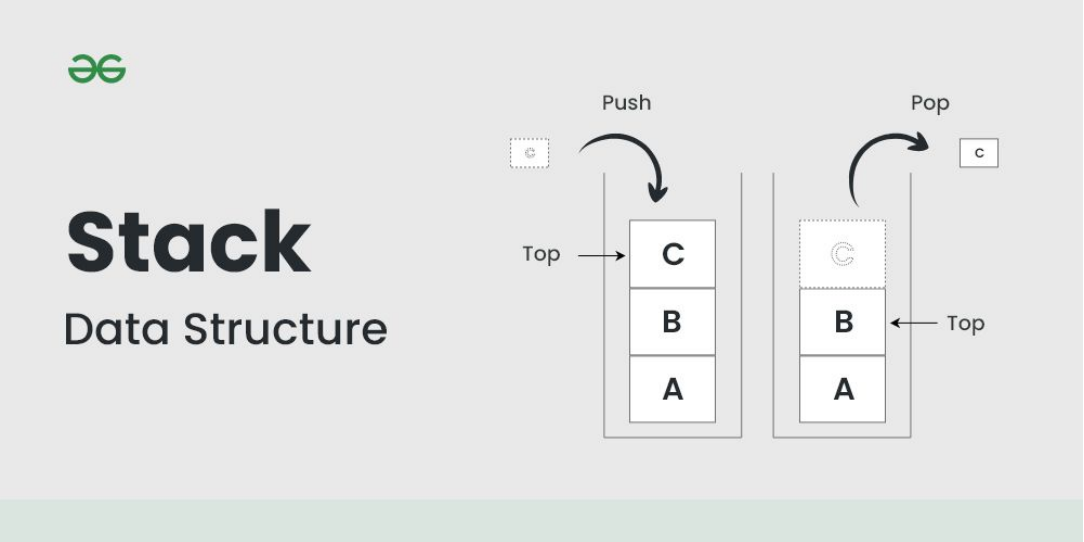
TDA Pila / ADT Stack



Si alguien necesita **un elemento**, lo saca del tope de la pila.

Cuando alguien **agrega un elemento** lo pone en el tope de la pila.

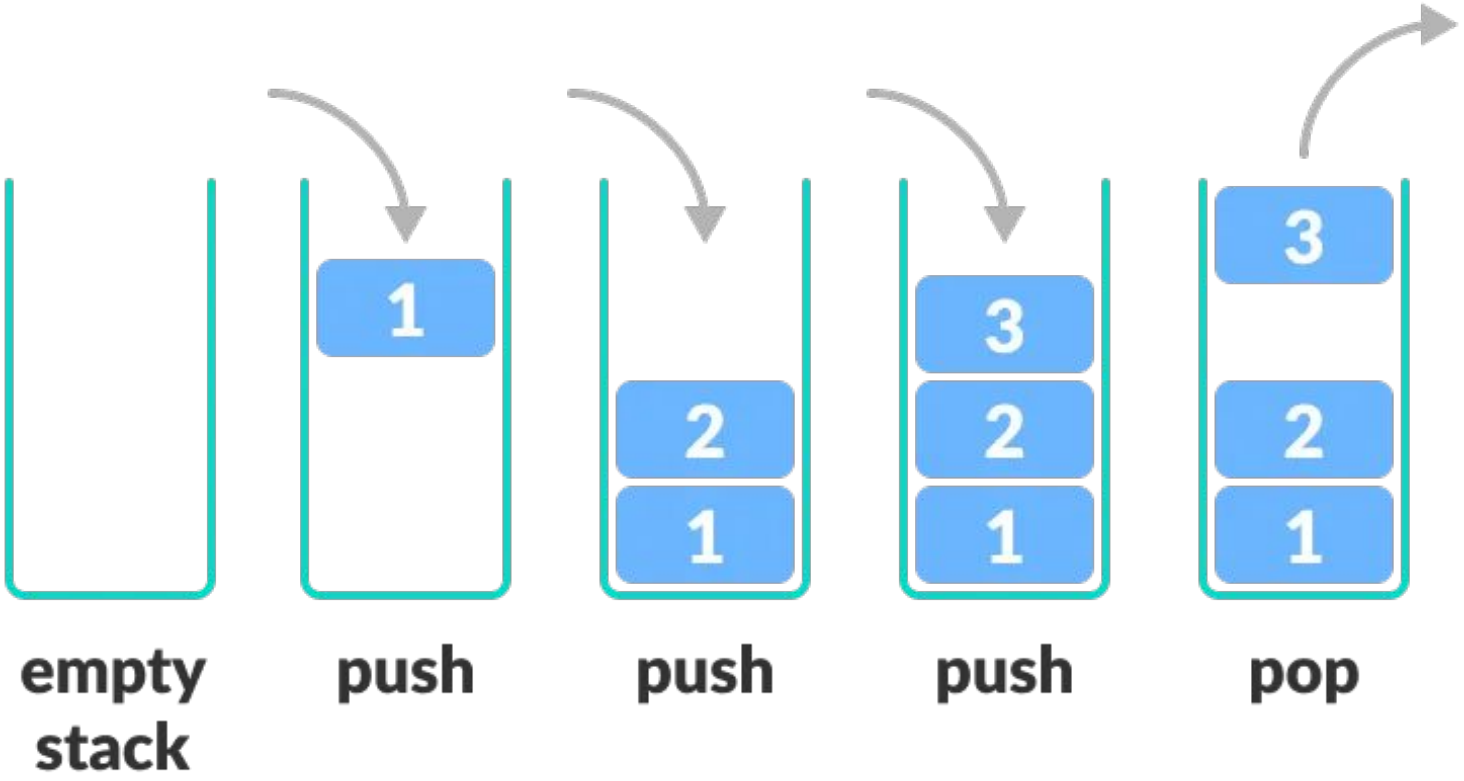
TDA Pila / ADT Stack



Si alguien necesita **un elemento**, lo saca del tope de la pila →***pop***

Cuando alguien **agrega un elemento** lo pone en el tope de la pila → ***push***

TDA Pila / ADT Stack



TDA Pila / ADT Stack

T = Tipo

D = Dato

A = Abstracto

Todo TDA se define en términos de su representación conceptual y sus operaciones (y de las restricciones que valen entre ellas)

TDA Pila / ADT Stack

Una pila es una estructura de datos lineal que sigue un orden particular en el que se realizan las operaciones. Este orden se denomina ***LIFO (last-in, first-out***). LIFO implica que el elemento que se inserta en último lugar sale primero.

TDA Pila / ADT Stack

push(e): Inserta el elemento e en el tope de la pila

pop(): Elimina el elemento del tope de la pila y lo entrega como resultado. Si se aplica a una pila vacía, produce una situación de error.

isEmpty(): Retorna verdadero si la pila no contiene elementos y falso en caso contrario.

top(): Retorna el elemento del tope de la pila. Si se aplica a una pila vacía, produce una situación de error.

size(): Retorna un entero natural que indica cuántos elementos hay en la pila.

TDA Pila / ADT Stack

OVERVIEW PACKAGE CLASS USE TREE DEPRECATED INDEX HELP

PREV CLASS NEXT CLASS FRAMES NO FRAMES ALL CLASSES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

compact1, compact2, compact3

java.util

Class Stack<E>

java.lang.Object
  java.util.AbstractCollection<E>
    java.util.AbstractList<E>
      java.util.Vector<E>
        java.util.Stack<E>

All Implemented Interfaces:

Serializable, Cloneable, Iterable<E>, Collection<E>, List<E>, RandomAccess

---

```
public class Stack<E>
extends Vector<E>
```

The Stack class represents a last-in-first-out (LIFO) stack of objects. It extends class Vector with five operations that allow a vector to be treated as a stack. The usual push and pop operatic stack, a method to test for whether the stack is empty, and a method to search the stack for an item and discover how far it is from the top.

When a stack is first created, it contains no items.

A more complete and consistent set of LIFO stack operations is provided by the Deque interface and its implementations, which should be used in preference to this class. For example:

```
Deque<Integer> stack = new ArrayDeque<Integer>();
```

Since:

JDK1.0

See Also:

Serialized Form

Field Summary

TDA Pila / ADT Stack

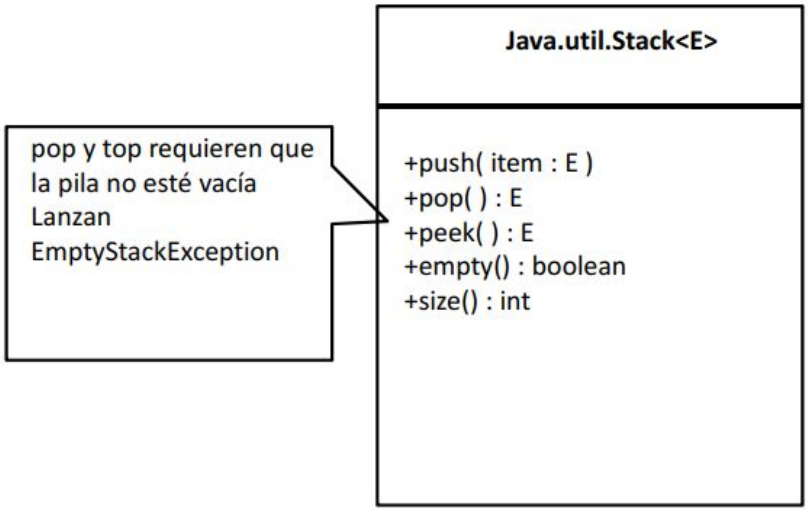
Method Summary

All Methods	Instance Methods	Concrete Methods

Modifier and Type	Method and Description
boolean	**empty**() Tests if this stack is empty.
E	**peek**() Looks at the object at the top of this stack without removing it from the stack.
E	**pop**() Removes the object at the top of this stack and returns that object as the value of this function.
E	**push**(E item) Pushes an item onto the top of this stack.
int	**search**(Object o) Returns the 1-based position where an object is on this stack.

Especificación de Stack en UML

En UML las operaciones se dan con una sintaxis Pascal-like y las excepciones se dan como comentarios:



Ejemplo de uso del TDA pila de la materia

```java
// Del repo de GitHub
public interface Stack<E> {
        /* ... */
        public int size();
        /* ... */
        public boolean isEmpty();
        /* ... */
        public E top();
        /* ... */
        public void push(E element);
        /**
         * Remueve el elemento que se encuentra en el tope de la pila.
         * @return Elemento removido.
         * @throws EmptyStackException si la pila está vacía.
         */
        public E pop();
}
```

¿Qué necesito ahora?

Asumamos que tengo una clase **StackSecreta** que implementa la interface **Stack**

Problema

- Desarrollar un método f que
  - Dada una pila de Strings, muestre por consola el elemento en el tope de la pila.
  - Si la pila está vacía mostrar un mensaje de error

¿Qué uso cómo argumento de mi método f?

- f (Stack<String> pila)
- f (StackSecreta<String> pila)

Problema

//Dada una pila de Strings, mostrar por consola el elemento en el tope de la pila si la pila está vacía mostrar un mensaje

```
public void f(Stack<String> pila) {




}
```

Problema

//Dada una pila de Strings, mostrar por consola el elemento en el tope de la pila si la pila está vacía mostrar un mensaje

```
public void f(Stack<String> pila) {
        if (pila.isEmpty()) {
                System.out.println("La pila esta vacía");
        }

}
```

Problema

//Dada una pila de Strings, mostrar por consola el elemento en el tope de la pila si la pila está vacía mostrar un mensaje

```
public void f(Stack<String> pila) {
	if (pila.isEmpty()) {
		System.out.println("La pila esta vacía");
	}
	else {
		String cadena = pila.top();
		System.out.println(cadena);
	}
}
```

Problema

//Dada una pila de Strings, mostrar por consola el elemento en el tope de la pila si la pila está vacía mostrar un mensaje

```
public void f(Stack<String> pila) {
        if (pila.isEmpty()) {
                System.out.println("La pila esta vacía");
        }
        else {
                String cadena = pila.top();
                System.out.println(cadena);
        }
}
```

¿Cómo llamo a f?

Supongamos que f está definido dentro de la clase MainParaPila

```
MainParaPila mpp = new MainParaPila();
Stack<String> mipila = new StackSecreta<String>();
mipila.push("Estructuras");
mipila.push("de");
mipila.push("Datos");
mpp.f(mipila);
```

TDA Cola / ADT Queue

TDA Cola / ADT Queue



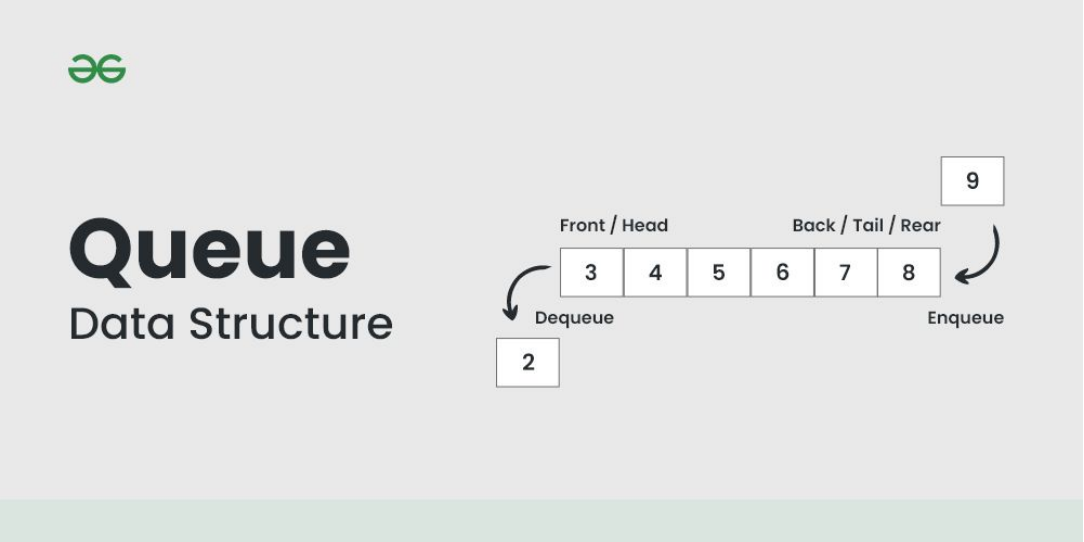
TDA Cola / ADT Queue



La primera persona en la cola, será la primera en ser atendida y saldrá de la cola.

La última persona en agregarse a la cola irá al final de la misma y será la última persona en salir de la cola.

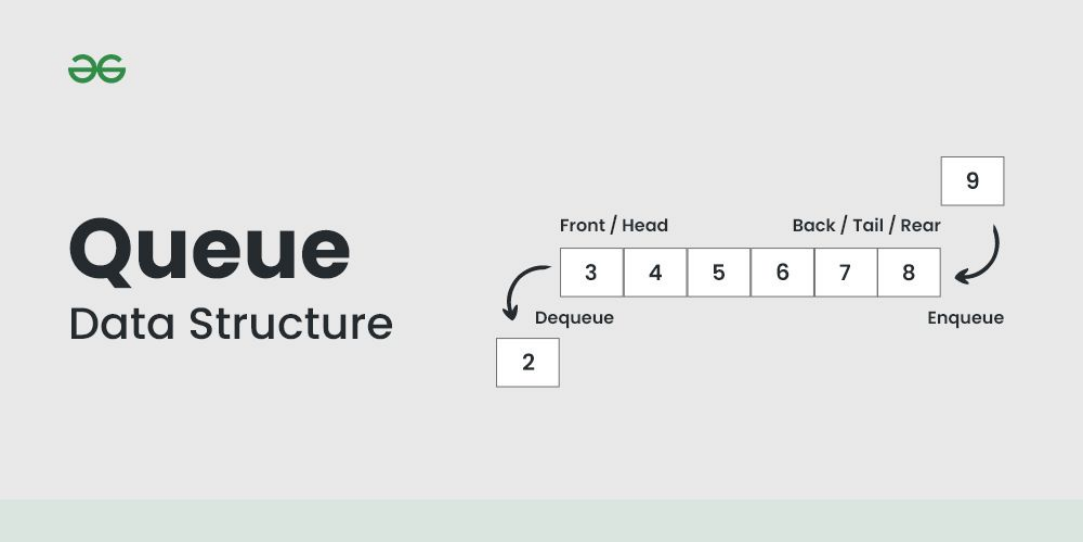
TDA Cola / ADT Queue



La primera persona en la cola, será la primera en ser atendida y saldrá de la cola.

La última persona en agregarse a la cola irá al final de la misma y será la última persona en salir de la cola.

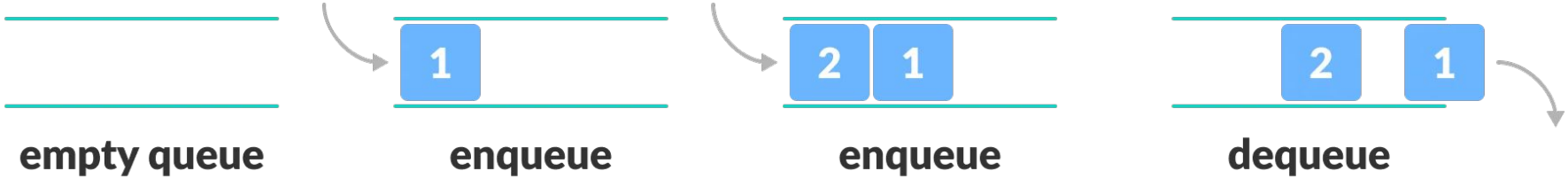
TDA Cola / ADT Queue



enqueue →agrega un elemento al final

dequeue → elimina un elemento del frente y lo retorna

TDA Cola / ADT Queue



TDA Cola / ADT Queue

La cola, al igual que la pila, es una estructura lineal. Lo que diferencia a la cola de la pila es que una cola está abierta en ambos extremos. Por lo tanto, sigue la estructura **FIFO (first-in, first-out)**. Se accede primero al elemento de datos insertado primero. Los datos se insertan en un extremo y se eliminan por el otro.

TDA Cola / ADT Queue

enqueue(e): Inserta el elemento e en el **back/tail/rear/rabo** de la cola

dequeue(): Elimina el elemento del frente de la cola y lo retorna. Si la cola está vacía se produce un error.

isEmpty(): Retorna verdadero si la cola no contiene elementos y falso en caso contrario.

front(): Retorna el elemento del frente de la cola. Si la cola está vacía se produce un error.

size(): Retorna un entero natural que indica cuántos elementos hay en la pila.

TDA Cola / ADT Queue

OVERVIEW PACKAGE CLASS USE TREE DEPRECATED INDEX HELP

PREV CLASS NEXT CLASS FRAMES NO FRAMES ALL CLASSES

SUMMARY: NESTED | FIELD | CONSTR | METHOD DETAIL: FIELD | CONSTR | METHOD

compact1, compact2, compact3
java.util

Interface Queue<E>

Type Parameters:
`E` - the type of elements held in this collection

All Superinterfaces:
`Collection<E>`, `Iterable<E>`

All Known Subinterfaces:
`BlockingDeque<E>`, `BlockingQueue<E>`, `Deque<E>`, `TransferQueue<E>`

All Known Implementing Classes:
`AbstractQueue`, `ArrayBlockingQueue`, `ArrayDeque`, `ConcurrentLinkedDeque`, `ConcurrentLinkedQueue`, `DelayQueue`, `LinkedBlockingDeque`, `LinkedBlockingQueue`
`SynchronousQueue`

```
public interface Queue<E>
extends Collection<E>
```

A collection designed for holding elements prior to processing. Besides basic `Collection` operations, queues provide additional insertion, extraction, and inspection opera
operation fails, the other returns a special value (either `null` or `false`, depending on the operation). The latter form of the insert operation is designed specifically for use
operations cannot fail.

Summary of Queue methods

	Throws exception	*Returns special value*
Insert	`add(e)`	`offer(e)`
Remove	`remove()`	`poll()`
Examine	`element()`	`peek()`

Queues typically, but do not necessarily, order elements in a FIFO (first-in-first-out) manner. Among the exceptions are priority queues, which order elements according to
which order the elements LIFO (last-in-first-out). Whatever the ordering used, the *head* of the queue is that element which would be removed by a call to `remove()` or `poll`
of queues may use different placement rules. Every `Queue` implementation must specify its ordering properties.

The `offer` method inserts an element if possible, otherwise returning `false`. This differs from the `Collection.add` method, which can fail to add an element only by throw
normal, rather than exceptional occurrence, for example, in fixed-capacity (or "bounded") queues.

The `remove()` and `poll()` methods remove and return the head of the queue. Exactly which element is removed from the queue is a function of the queue's ordering policy
methods differ only in their behavior when the queue is empty: the `remove()` method throws an exception, while the `poll()` method returns `null`.

TDA Cola / ADT Queue

Method Summary

All Methods | **Instance Methods** | **Abstract Methods**

Modifier and Type	Method and Description
boolean	**add**(E e) Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions, returning `true` upon success and throwing an `IllegalStateException` if no space is currently available.
E	**element**() Retrieves, but does not remove, the head of this queue.
boolean	**offer**(E e) Inserts the specified element into this queue if it is possible to do so immediately without violating capacity restrictions.
E	**peek**() Retrieves, but does not remove, the head of this queue, or returns `null` if this queue is empty.
E	**poll**() Retrieves and removes the head of this queue, or returns `null` if this queue is empty.
E	**remove**() Retrieves and removes the head of this queue.

Ejemplo de uso del TDA cola de la materia

```
// Del repo de GitHub
public interface Queue<E> {
        /* ... */
        public int size();
        /* ... */
        public boolean isEmpty();
        /* ... */
        public E front();
        /* ... */
        public void enqueue(E element);
        /**
         * Remueve el elemento que se encuentra en el frente de la cola.
         * @return Elemento removido.
         * @throws EmptyStackException si la pila está vacía.
         */
        public E dequeue();
}
```

¿Qué necesito ahora?

Asumamos que tengo una clase **QueueSecreta** que implementa la interface **Stack**

Problema

//Dada una cola de Strings, mostrar por consola el elemento en el fondo de la cola. Si la cola está vacía mostrar un mensaje

```
public void g(Queue<String> cola) {
	if (cola.isEmpty()) {
		System.out.println("La cola esta vacía");
	}
	else {

	}
}
```

Problema

//Dada una cola de Strings, mostrar por consola el elemento en el fondo de la cola. Si la cola está vacía mostrar un mensaje

```
public void g(Queue<String> cola) {
        if (cola.isEmpty()) {
                System.out.println("La cola esta vacía");
        }
        else {
                String cadena;
                while (! cola.isEmpty()) {
                        cadena = cola.dequeue();
                }
                System.out.println(cadena);
        }
}
```

¿Qué problema tiene g?

Destruyo el contenido de la estructura!

Problema

//Dada una cola de Strings, mostrar por consola el elemento en el fondo de la cola. Si la cola está vacía mostrar un mensaje

```
public void g(Queue<String> cola) {
        if (cola.isEmpty()) {
                System.out.println("La cola esta vacía");
        }
        else {
                Queue<String> auxiliar = new QueueSecreta<String>();
                String cadena;
                while (! cola.isEmpty()) {
                        cadena = cola.dequeue();
                        auxiliar.enqueue(cadena);
                }
                System.out.println(cadena);
        }
}
```

¿Terminé?

Problema

//Dada una cola de Strings, mostrar por consola el elemento en el fondo de la cola.
Si la cola está vacía mostrar un mensaje

```
public void g(Queue<String> cola) {
        if (cola.isEmpty()) {
                System.out.println("La cola esta vacía");
        }
        else {
                Queue<String> auxiliar = new QueueSecreta<String>();
                String cadena;
                while (! cola.isEmpty()) {
                        cadena = cola.dequeue();
                        auxiliar.enqueue(cadena);
                }
                System.out.println(cadena);
                while (! auxiliar.isEmpty()) {
                        cadena = auxiliar.dequeue();
                        cola.enqueue(cadena);
                }
        }
}
```

¿Cómo llamo a g?

Supongamos que g está definido dentro de la clase MainParaCola

```
MainParaCola mpc = new MainParaCola();
Queue<String> miQueue = new QueueSecreta<String>();
miQueue.push("Estructuras");
miQueue.push("de");
miQueue.push("Datos");
mpc.g(miQueue);
```

TDA Cola & TDA Pila

TDA Cola & TDA Pila

¿Cómo invertir una Cola usando sólo los TDAs vistos?

¿Cómo invertir una Pila usando sólo los TDAs vistos?



THE END

Bibliografía

M.T. Goodrich & R. Tamassia. Data Structures and Algorithms in Java. Fourth Edition. John Wiley & Sons, 2006.

M.T. Goodrich, R. Tamassia, Michael Goldwasser. Data Structures and Algorithms in Java. Sixth Edition. John Wiley & Sons, 2014.